Programação Avançada

Unidade 1: Programação Orientada a Objetos (POO)

Aula 05: Programação Orientada a Objetos – Herança e Polimorfismo

Prof. Eduardo Corrêa Gonçalves

30/03/2026

Sumário

Introdução

O que é Herança?

Implementando a Herança

Instanciando uma Subclasse

Sobrescrevendo Métodos

Estendendo uma Classe

Herança Múltipla

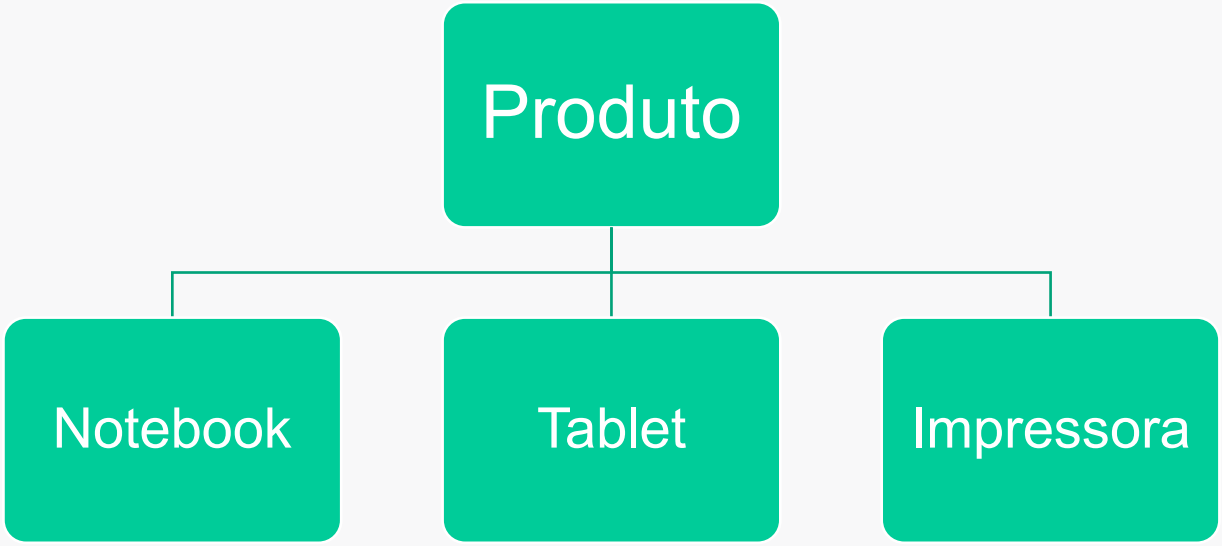
Polimorfismo

Mergulhando nos Detalhes

Sobrecarga de Operadores Utilizando Métodos Mágicos

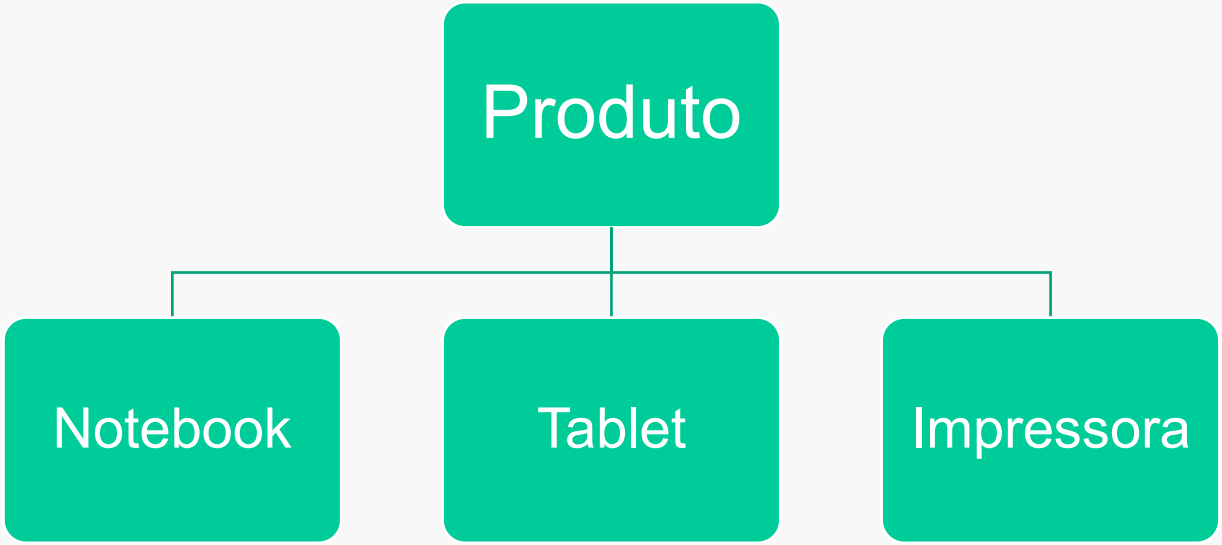
Introdução (1/6)

- **O que é Herança?**
  - Trata-se de um mecanismo utilizado na POO para **derivar novas classes** a partir de **classes já existentes**.
  - Com isso torna-se possível criar uma **hierarquia** de classes.
    - **Exemplo**: a partir da classe Produto, poderíamos derivar as classes Notebook, Tablet e Impressora



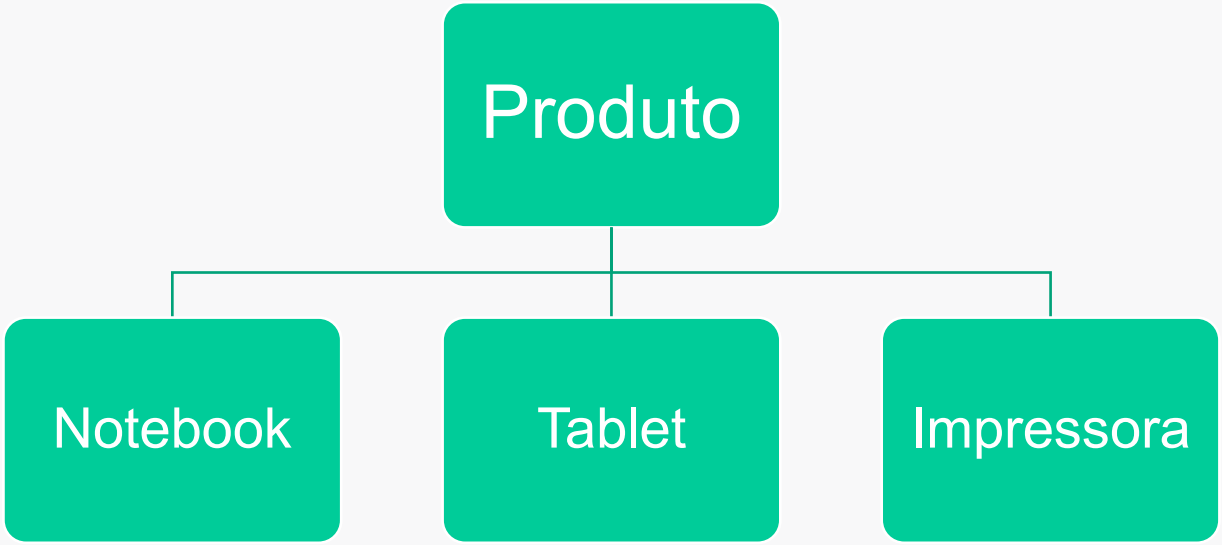
Introdução (2/6)

- **Herança – Ideia Básica:**
  - Possibilitar com que o desenvolvedor crie uma **nova classe** (**classe filho** ou **subclasse**) a partir de uma **classe já existente** (**classe pai** ou **superclasse**).
    - **Exemplo**: criar a classe Notebook a partir de Produto e não “do zero”
    - Isto habilita a classe filho a **herdar** os atributos e métodos da classe da classe pai



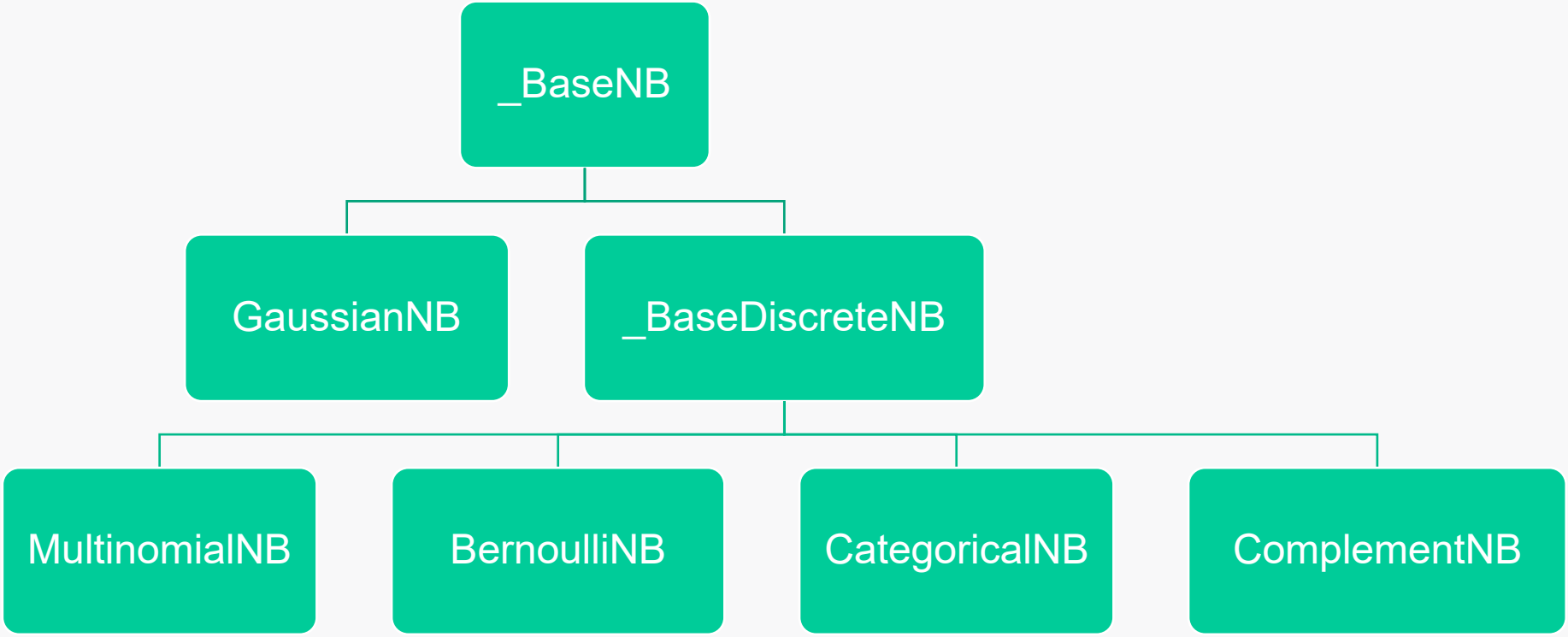
Introdução (3/6)

- **Herança - Benefícios esperados**:
  - Prover **reutilização de código**.
  - Permitir com que códigos escritos previamente sejam **estendidos** de forma fácil para satisfazer as necessidades de uma nova situação.
  - Possibilitar uma melhor **modelagem das relações** existentes entre as classes de um minimundo
    - Isto torna o código mais elegante e o sistema melhor documentado.



Introdução (4/6)

- **Exemplo real – Implementação do Naïve Bayes na scikit-learn**
  - A biblioteca oferece 5 versões do Naïve Bayes (NB): **GaussianNB**, **BernoulliNB**, **CategoricalNB**, **MultinomialNB** e **ComplementNB**.
    - **_BaseNB** oferece as funcionalidades comuns a todas.
    - **_BaseDiscreteNB** oferece as funcionalidades comuns às versões do NB que trabalham com variáveis discretas.
      - A redução da quantidade de código necessária para implementar as 5 versões do NB é imensa!!!



Introdução (5/6)

- **Exemplo desta aula**
  - Nesta aula, veremos como implementar a herança no Python.
  - Criaremos uma nova classe chamada **NurseRobot** (robô enfermeiro) a partir da classe **Robot** (robô comum).
  - Considere que NurseRobot é um robô utilizado para recarregar outros robôs que estejam com pouca energia (“doentes”).
    - Ou seja, um NurseRobot possui tudo que um robô comum tem, mais a capacidade de recarregar outros robôs.

- Baseado no exemplo de: https://www.python-course.eu/python3_inheritance.php

Introdução (6/6)

- **Exemplo desta aula**
  - A sintaxe para a definição de uma subclasse é a seguinte:

```
class NurseRobot(Robot):
    pass
```

  - Basta colocar entre parênteses o nome da classe pai.
  - Nos próximos slides, vamos implementar essa subclasse e examinar diversas caraterísticas da técnica de herança.

Herança (1/14)

- **A classe pai: Robot**
  - Duas propriedades: “_nome” e “_ano_fabricacao"
  - Um método: “falar_oi()”
    - Digite e salve no arquivo “robot.py”

robot.py

```
class Robot:

    def __init__(self, nome, ano_fabr):
        self._nome = nome
        self._ano_fabricacao = ano_fabr

    def falar_oi(self):
        print("oi, sou {} de {}".format(self._nome, self._ano_fabricacao))
```

Herança (2/14)

- **A classe filho: NurseRobot**
  - Para começar, ela será criada da forma abaixo (sem atributos ou métodos).
    - Digite e salve como “nurse_robot.py”

nurse_robot.py

```
from robot import Robot

class NurseRobot(Robot):
    pass
```

- **Obs.:** Estamos salvando as classes pai e filho em arquivos separados, mas poderíamos ter definido ambas em um mesmo arquivo.
  - Nada impede que sejam definidas várias classes em um único módulo.

Herança (3/14)

- **Instanciado um objeto NurseRobot**
  - Embora não tenhamos definido nenhum atributo ou método em NurseRobot, os objetos dessa classe herdam automaticamente:
    - **Todos os atributos** de Robot (mesmo que sejam protegidos);
    - **Todos os métodos** de Robot (incluindo o construtor).

```
>>> enf = NurseRobot("Florence", 2019)
>>> enf.falar_oi()
oi, sou Florence de 2019
>>>
>>> enf._nome
'Florence’
>>> enf._ano_fabricacao
2019
```

Herança (4/14)

- **Instanciado um objeto NurseRobot**
  - Criar uma nova classe com definição idêntica não parece ser tão útil...

```
>>> enf = NurseRobot("Florence", 2019)
>>>
>>> enf.__dict__
{'_nome': 'Florence', '_ano_fabricacao': 2019}
```

  - De fato, o que torna a herança realmente uma ferramenta indispensável na POO são algumas técnicas relacionadas, em especial a **sobrescrita** e a **extensão**.

Herança (5/14)

- **Sobrescrita (*overriding*)**
  - Em nosso minimundo, um objeto NurseRobot tem várias similaridades com Robot, mas não é exatamente igual.
  - Por exemplo, já que NurseRobot é um enfermeiro, talvez seja legal que ele se apresente (fale oi) de forma diferente.
    - Para conseguir isso, podemos **sobrescrever** (ou reescrever) o método “falar_oi()” dentro da subclasse NurseRobot.

nurse_robot.py

```
from robot import Robot

class NurseRobot(Robot):

    def falar_oi(self):
        print("oi, sou {} de {}".format(self._nome, self._ano_fabricacao))
        print("Sou especialista em enfermagem!")
```

Herança (6/14)

- **Sobrescrita (*overriding*)**
  - Agora um objetos Robot e NurseRobot falam “oi” de forma diferente.

```
>>> z = Robot("Torg",2008)
>>> z.falar_oi()
oi, sou Torg de 2008
>>>
>>> enf = NurseRobot("Florence",2019)
>>> enf.falar_oi()
oi, sou Florence de 2019
Sou especialista em enfermagem!
```

  - Com a sobrescrita, conseguimos especializar os métodos herdados das superclasses, **alterando o seu comportamento** nas subclasses por um mais específico.
    - Um método de uma classe pai é sobrescrito através da simples definição de um **método com o mesmo nome** na classe filho.

Herança (7/14)

- **Sobrescrita – super()**
  - Para não colocar código redundante na classe filho podemos usar a função **super()**.
    - Essa função faz uma chamada para o método na classe pai.

nurse_robot.py

```
from robot import Robot

class NurseRobot(Robot):

    def falar_oi(self):
        super().falar_oi()
        print("Sou especialista em enfermagem!")
```

- **super().**falar_oi() faz com que o código do método falar_oi() de Robot seja executado.
  - Também seria possível usar Robot.falar_oi(self).

Herança (8/14)

- **Estendendo uma classe**
  - A classe filho pode estender a classe pai.
  - Isto é, eu posso **criar novos métodos e atributos** na classe filho.
    - Estes estarão disponíveis apenas para os objetos da classe filho.
      - Afinal de contas, a ideia é que se tratem de dados ou comportamentos específicos para objetos da subclasse.
    - Na maioria das situações práticas, quase sempre precisamos criar novos atributos ou métodos nas subclasses.

Herança (9/14)

- **Estendendo uma classe – exemplo 1**
  - No exemplo abaixo, a subclasse NurseRobot estende Robot com um novo método: “recarregar()”.
    - Considere que ele é usado para um NurseRobot recarregar a bateria de um Robot.

```
from robot import Robot
import time

class NurseRobot(Robot):

    def falar_oi(self):
        super().falar_oi()
        print("Sou especialista em enfermagem!")

    def recarregar(self, outro_robo):
        print(self._nome + " recarregando " + outro_robo._nome)
        for x in range(5):
            print(".", end=" ")
            time.sleep(1)
        print("CARGA COMPLETA!")
```

nurse_robot.py

Herança (10/14)

- **Estendendo uma classe – exemplo 1**

```
>>> z = Robot("Torg",2008)
>>>
>>> enf = NurseRobot("Florence",2019)
>>> enf.recarregar(z)
Florence recarregando Torg
. . . . . CARGA COMPLETA!
```

Herança (11/14)

- **Estendendo uma classe – exemplo 2**
  - No exemplo abaixo, a definição de NurseRobot:
    - Sobrescreve o construtor de Robot;
    - E, ao mesmo tempo, estende a classe pai com um novo atributo chamado **_num_cre** (*número do registro do NurseRobot no conselho regional de enfermagem...*)

```
from robot import Robot

class NurseRobot(Robot):

    def __init__(self, nome, ano_fabr, num_cre):
        super().__init__(nome, ano_fabr)
        self._num_cre = num_cre

    def falar_oi(self):
        super().falar_oi()
        print("Sou especialista em enfermagem!")
        print("Meu CRE é: ", self._num_cre)
```

nurse_robot.py

Herança (12/14)

- **Estendendo uma classe – exemplo 2**

```
>>> enf = NurseRobot("Florence",2019,123456)
>>> enf.falar_oi()
oi, sou Florence de 2019
Sou especialista em enfermagem!
Meu CRE é:  123456
>>>
>>> enf.__dict__
{'_nome': 'Florence', '_ano_fabricacao': 2019, '_num_cre': 123456}
```

Herança (13/14)

- **Herança Múltipla**
  - O Python também suporta que uma subclasse herde de mais de uma classe pai.
    - No exemplo abaixo, a classe M é derivada de B1 e B2.
    - Isso significa que ela herdará todos os atributos/métodos de B1 e B2

```
class B1:
    pass

class B2:
    pass

class M(B1, B2):
    pass
```

Herança (14/14)

- **Herança Múltipla**
  - Problema: possibilidade de conflito!
    - Atributos ou métodos com o mesmo nome nas classes pais.

```
class B1:
    pass

class B2:
    pass

class M(B1, B2):
    pass
```

  - **Resolução**: todos os atributos / métodos são procurados primeiro na definição de M e depois nas superclasses considerando a ordem de especificação (no exemplo, primeiro em B1 e depois em B2).

  - Mais detalhes em: https://realpython.com/lessons/multiple-inheritance-python/

Polimorfismo (1/4)

- **Definição**
  - Na herança, vimos que todo NurseRobot é um Robot, pois é uma extensão deste.
  - Então, se alguém precisar de um Robot, normalmente poderá pegar um NurseRobot, sem que isso gere problemas!
    - Por que? Porque NurseRobot **é um** Robot...
    - O NurseRobot faz tudo que o Robot faz e mais um pouco!
    - Essa é a semântica da herança.
  - Este exemplo ilustra o **polimorfismo**, a capacidade de um objeto de ser referenciado de várias formas.

Polimorfismo (2/4)

- **Exemplo**
  - No exemplo abaixo, definimos a classe Crianca.
  - O método brincar(), serve para fazer uma instância de criança brincar com uma instância de Robot.

crianca.py

```
class Crianca:

    def __init__(self, nome):
        self._nome = nome

    def brincar(self, robo):
        print("Vou brincar com ", robo._nome)
```

Polimorfismo (3/4)

- **Exemplo**
  - Veja que podemos passar tanto um Robot como um NurseRobot para um método que recebe um “Robot” como entrada...

```
from robot import Robot
from nurse_robot import NurseRobot
from crianca import Crianca

z = Robot("Torg",2008)
enf = NurseRobot("Florence",2019, 123456)
c = Crianca("Joãozinho")
c.brincar(z)
c.brincar(enf)
>>> %Run testa_crianca.py
Vou brincar com Torg
Vou brincar com Florence
```

Polimorfismo (4/4)

- **type() x isinstance()**
  - Ambas as funções acima são utilizadas para checar o tipo de um objeto. Porém, há uma pequena diferença:
    - **isinstance()**: retorna True se comparamos o objeto com a sua classe real ou sua superclasse.
    - **type()**: utiliza o comparador == e só retorna True se comparamos objeto com a sua classe real.

```
>>> enf = NurseRobot("Florence",2019, 123456)
>>> isinstance(enf, Robot)
True
>>> isinstance(enf, NurseRobot)
True
>>> type(enf)==Robot
False
>>> type(enf)==NurseRobot
True
```

Métodos Mágicos (1/9)

- **Definição**
  - Os métodos mágicos são aqueles de nomes estranhos, com dois underscores "__" no início e no fim.
  - Nós já trabalhamos com dois deles em aulas anteriores: __init__() e __str__()
  - Porém, existem muitos outros, como __add__(), __eq__(), __gt__(), __hash__(), ...
  - E **por que** os métodos mágicos são **úteis**?
    - Porque possibilitam com que a gente trabalhe com os objetos de nossas classes de forma semelhante a dos objetos nativos do Python

Métodos Mágicos (2/9)

- **Sobrecarga (*overload*)**
  - Por exemplo, suponha que eu deseje comparar dois robôs r1 e r2 com o uso do operador maior que “ > ”.
  - Vamos considerar que r1 > r2, caso r1 tenha sido fabricado depois de r2.
    - Dessa forma, nosso critério é o ano de fabricação.
    - Se um robô é mais “novo” que outro, vou considerar que ele é “maior”.
  - Veja que ocorre **erro** se tento utilizar o operador “>” diretamente:

```
>>> r1 = Robot("Hal",1968)
>>> r2 = Robot("Zed", 2020)
>>> r2 > r1
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
TypeError: '>' not supported between instances of 'Robot' and 'Robot'
```

Métodos Mágicos (3/9)

- **Sobrecarga (*overload*)**
  - Eu posso resolver esta questão utilizando o método mágico __gt__()
    - Caso eu crie um método com este nome na classe Robot, ele será automaticamente chamado toda vez que eu utilizar o operador “ > “
    - Isto é chamado de **sobrecarga** do operador “ > ”

```
class Robot:

    def __init__(self, nome, ano_fabr):
        self._nome = nome
        self._ano_fabricacao = ano_fabr

    def falar_oi(self):
        print("oi, sou {} de {}".format(self._nome, self._ano_fabricacao))

    def __gt__(self, outro_robo):
        if self._ano_fabricacao > outro_robo._ano_fabricacao:
            return True
        else:
            return False
```

Métodos Mágicos (4/9)

- **Sobrecarga (*overload*)**
  - Veja que agora, nossa classe sabe fazer a comparação utilizando o operador “ > ” e seguindo o critério que definimos em nossa classe.

```
>>> r1 = Robot("Hal",1968)
>>> r2 = Robot("Zed", 2020)
>>> r2 > r1
True
>>> r1 > r2
False
```

Métodos Mágicos (5/9)

- O método __gt__() é chamado de mágico, pois é chamado automaticamente quando usamos “>” em uma expressão.
- Da mesma forma, poderíamos ter sobrecarregado os operadores:
  - == com __eq__()
  - ≠ com __ne__()
  - < com __lt__()
  - ≥ com __ge__()
  - ≤ com __le__()
  - ...
- Veja na página a seguir a relação com os métodos mágicos mais usados.
- Para informações detalhadas, consulte:
  - https://www.python-course.eu/python3_magic_methods.php

Métodos Mágicos (6/9)

- **Relação com os principais métodos mágicos** (*Lee & Hubbard, 2015*)
  - Nesta tabela, self e x referem-se ao mesmo objeto.

Method Defintion	Operator	Description
__add__(self,y)	x + y	The addition of two objects. The type of *x* determines which add operator is called.
__contains__(self,y)	y in x	When *x* is a collection you can test to see if *y* is in it.
__eq__(self,y)	x == y	Returns *True* or *False* depending on the values of *x* and *y*.
__ge__(self,y)	x >= y	Returns *True* or *False* depending on the values of *x* and *y*.
__getitem__(self,y)	x[y]	Returns the item at the yth position in x.
__gt__(self,y)	x > y	Returns *True* or *False* depending on the values of *x* and *y*.
__hash__(self)	hash(x)	Returns an integral value for *x*.
__int__(self)	int(x)	Returns an integer representation of *x*.
__iter__(self)	for v in x	Returns an iterator object for the sequence *x*.
__le__(self,y)	x <= y	Returns *True* or *False* depending on the values of *x* and *y*.
__len__(self)	len(x)	Returns the size of *x* where *x* has some length attribute.
__lt__(self,y)	x < y	Returns *True* or *False* depending on the values of *x* and *y*.
__mod__(self,y)	x % y	Returns the value of *x* modulo *y*. This is the remainder of *x/y*.
__mul__(self,y)	x * y	Returns the product of *x* and *y*.
__ne__(self,y)	x != y	Returns *True* or *False* depending on the values of *x* and *y*.
__neg__(self)	-x	Returns the unary negation of *x*.
__repr__(self)	repr(x)	Returns a string version of x suitable to be evaluated by the *eval* function.
__setitem__(self,i,y)	x[i] = y	Sets the item at the ith position in *x* to *y*.
__str__(self)	str(x)	Return a string representation of *x* suitable for user-level interaction.
__sub__(self,y)	x - y	The difference of two objects.

Métodos Mágicos (7/9)

- **Mais um exemplo: __add__()**
  - O método __add__() serve para executar a adição de dois objetos.
  - **Por exemplo**: se realizamos uma soma de inteiros no Python, como 10 + 5, o que o interpretador faz é chamar o método __add__() da classe int.

```
>>> x = 10
>>> x.__add__(5)
15
```

  - Imagine agora que você estivesse implementando uma classe Vetor, para representar um vetor da Geometria Analítica.
    - Você poderia implementar a adição dos vetores em __add__()
    - Assim, utilizaria a sintaxe w = u + v, para somar dois vetores u e v, armazenando o vetor resultante em w.

Métodos Mágicos (8/9)

- **Outro exemplo: __eq__()**
  - Quando criamos uma nova classe, há alguns métodos mágicos que ficam automaticamente disponíveis, porém com um **comportamento default**.
  - Este é o caso de __eq__(), que corresponde ao == (comparação).
    - Em seu comportamento default, ele dirá que dois objetos de uma classe são iguais, apenas se armazenarem o mesmo id (endereço de memória)

foo.py

```
class Foo:

    def __init__(self, n):
        self.n = n
```

Shell

```
>>> %Run foo.py
>>> a = Foo(1)
>>> b = Foo(1)
>>> a is b
False
>>> a == b
False
```

Métodos Mágicos (9/9)

- **Outro exemplo: __eq__()**
  - Para alterar esse comportamento, basta implementar o método __eq__() em Foo.
  - Assim podemos determinar explicitamente como comparar os nossos objetos.

foo.py

```
1  class Foo:
2
3      def __init__(self, n):
4          self.n = n
5
6      def __eq__(self, other):
7          if self.n == other.n:
8              return True
9          else:
10             return False
```

Shell

```
>>> a = Foo(1)
>>> b = Foo(1)
>>> a is b
False

>>> a == b
True
```

Considerações Finais

- **Herança**
  - Princípio usado na POO que consiste em permitir a definição de novas classes (classes filho) a partir de uma classe já existente (classe pai).
  - **Benefícios**:
    - Redução do tamanho do código
      - Posso deixar tudo que é comum na classe pai e isso automaticamente será herdado pelos filhos.
    - Melhor organização do sistema;
      - Evidencia os relacionamentos entre as classes.
    - Permite estender código já existente.
      - Facilita a implementação de uma nova funcionalidade que seja muito similar a alguma outra já desenvolvida.

Tarefa

(1) - Implemente uma classe chamada **Retangulo** que armazene as coordenadas de um retângulo, dadas por (x1, y1) e (x2, y2).

- Todos os pontos devem pertencer ao 1o quadrante, isto é x1, x2, y1, y2 ≥ 0. Considere ainda que x2 > x1 e y2 > y1. Se alguma dessas condições for violada, imprima a mensagem “Entrada Incorreta”.
- Implemente os métodos largura() e altura() para retornar, respectivamente, a largura e a altura do retângulo.
- Implemente o método area() para retornar a área do retângulo
- Implemente o método perímetro() para retornar o perímetro do retângulo.
- Implemente o método __str__() para que na impressão de um objeto do tipo retângulo, as suas coordenadas sejam apresentadas : “(x1, y1) e (x2, y2)”

(2) - Crie uma subclasse **Quadrado** com base na superclasse Retangulo.

- Implemente o construtor de Quadrado. Este deverá receber apenas três parâmetros: “x1”, “y1” e “lado”. De acordo com esses argumentos, você deverá invocar o construtor de Retangulo usando super(). Além do construtor, Quadrado não precisará sobrescrever nenhum método ou atributo de Retangulo.
- Instancie dois objetos do tipo Quadrado, invoque os métodos area() e perimetro() e imprima os objetos com print(). Verifique se os cálculos estão retornando valores corretos e se as coordenadas dos quadrados estão consistentes com o tamanho do “lado” passado como argumento.

Referências

- Python 3 Tutorial – Inheritance
  https://www.python-course.eu/python3_inheritance.php

- Python 3 Tutorial – Magic Methods and Operator Overloading
  https://www.python-course.eu/python3_magic_methods.php

- CAELUM (2020). Python e Orientação a Objetos – Curso PY-14. (*Cap. 10*).

- Trudeau, C. (2020). Multiple Inheritance in Python.
  https://realpython.com/lessons/multiple-inheritance-python/

- Stack Overflow (2009). “Elegant ways to support equivalence (“equality”) in Python classes”.
  https://stackoverflow.com/questions/390250/elegant-ways-to-support-equivalence-equality-in-python-classes